02/03/25 Day 7

Resource ownership concept

Resource

- We can say `Resource` is anything that has a cost associated with it and needs to be managed properly to avoid issues such as memory leaks, resource exhaustion, or unintended side effects.
- Resource is any entity that requires management and has a lifecycle, such as:
  - Memory
  - File handles
  - Network connections
  - Database connections

Resource management

- The process of efficiently and effectively handling resources to ensure they are allocated, used, and released properly.
- It involves techniques and strategies to prevent resource leaks, optimize performance, and ensure the stability of applications.

Resource ownership

- The concept of resource ownership refers to the responsibility of managing and controlling access to resources.

RAII (Resource Acquisition Is Initialization)

- RAII is a programming idiom used in C++ to manage resources.
- It ensures that resources are properly acquired and released by tying their lifecycle to the scope of an object.

> `Resource Owner` owns the resource is responsible for its management, including allocation, usage, and deallocation. `Resource Owner`(function, class, scope, etc.) acquires the resource is responsible for releasing it.

Static data members

- Static data members are shared among all instances of a class, meaning they have the same value for every object of that class.
- They are declared using the `static` keyword and are typically used to store information that is common to all objects of the class, such as a count of the number of instances created or a shared configuration setting.
- Static data members are accessed using the class name rather than through an instance of the class.

```
class className {
public:
```

```
    static int staticMember; // Static data member declaration
};
```

Static member functions

- Static member functions are functions that belong to the class rather than to any specific instance of the class.
- They are declared using the `static` keyword and can be called without creating an instance of the class.
- Static member functions can only access static data members and other static member functions of the class.
- `this` pointer is not available in static member functions because they are not associated with any instance of the class.

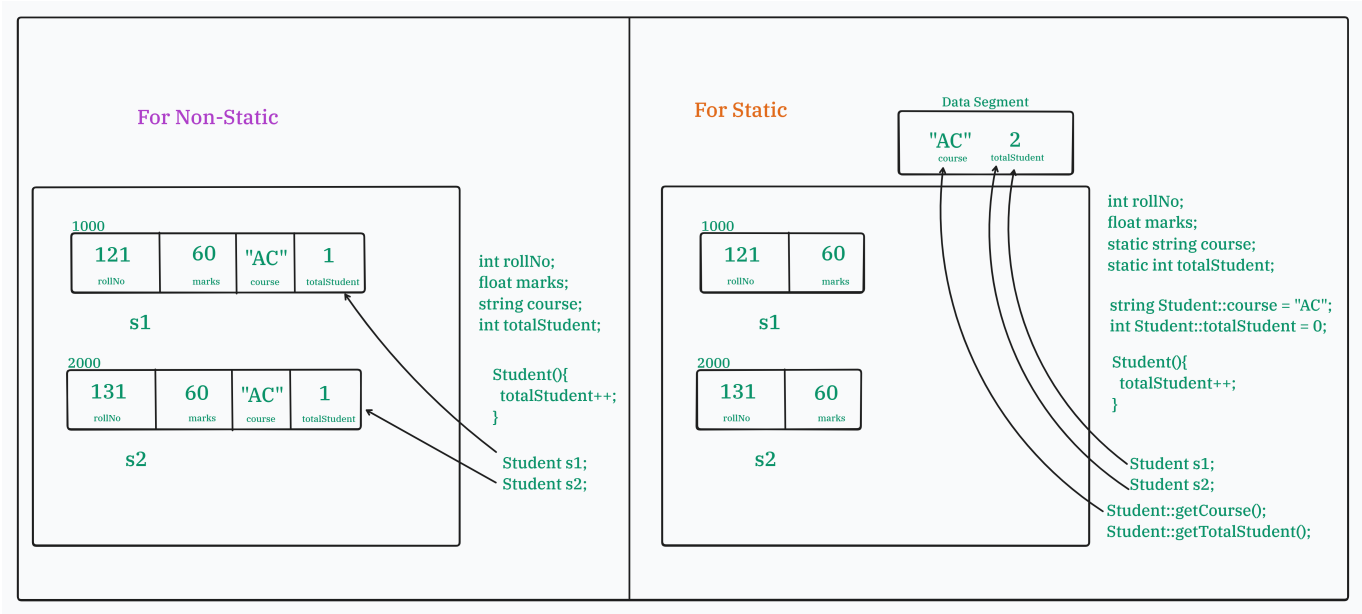
> Static member functions cannot access non-static members of the class because they do not have access to the `this` pointer, which is used to refer to the current instance of the class.

```
class className {
public:
    static void staticFunction() { // Static member function declaration
        // Function body
    }
};
```

```
#include <iostream>
using namespace std;

class Student{
    int rollNo;
    float marks;
    static string course;   // static data member declaration
    static int totalStudent;    // static data member declaration

public:

    Student() : rollNo(0), marks(0){
        totalStudent++;
    }

    Student(int rollNo, float marks) : rollNo(rollNo), marks(marks){
        totalStudent++;
    }

    float getMarks() const {
        return marks;
    }

    void setMarks(float marks) {
```

```
        this->marks = marks;
    }

    int getRollNo() const {
        return rollNo;
    }

    void setRollNo(int rollNo) {
        this->rollNo = rollNo;
    }

    //static member function
    static const string& getCourse() {
        return course;
    }

    void setCourse(const string &course) {
        this->course = course;
    }

    static int getTotalStudent() {
        return totalStudent;
    }

    void setTotalStudent(int totalStudent) {
        this->totalStudent = totalStudent;
    }

    // static member function to display total students and course
    static void getStudentsAndCourse(){
        cout << "Total Student: " << totalStudent << " | Course: " << course <<
endl;
    }
};

string Student::course = "PGCP-AC";
int Student::totalStudent = 0;

int main() {

    Student s1(121, 48);

    cout << "Total Student : "<< Student::getTotalStudent() << endl; // static

    Student s2(132, 67);

    Student s3;

    cout << "Total Student : "<< Student::getTotalStudent() << endl;
                                                            // s1.getCourse() :
not recommended
    cout << s1.getRollNo() << " | " << s1.getMarks() << " | "<<
Student::getCourse() << endl;
    cout << s2.getRollNo() << " | " << s2.getMarks() << " | "<<
```

```
Student::getCourse() << endl;

    Student::getStudentsAndCourse();

    return 0;
}
```



Friend function

- A friend function is a function that is not a member of a class but has access to the private and protected members of the class.
- It is declared using the `friend` keyword inside the class definition.
- Friend functions can be useful when you want to allow a non-member function to access the private members of a class without making those members public.

```
#include <iostream>
using namespace std;

class Student{
    int rollNo;
    string name;
    float marks;
public:

    Student(): rollNo(0), name(""), marks(0){
    }

    Student(int rollNo, string name, float marks): rollNo(rollNo), name(name),
marks(marks){
    }

    friend void displayStudent(Student&);   // friend function
};
```

```
// global function which is a friend of Student class
void displayStudent(Student& s){
    cout << s.rollNo << " | " << s.name << endl;
}

int main() {

    Student s1(121, "shil", 45);

//  cout << s1.getRollNo() << " | " << s1.getName() << endl;

    displayStudent(s1); //call to global function which is a friend of Student
class

    return 0;
}
```

Friend class

```
#include <iostream>
using namespace std;

//class Student;    // forward declaration

class Student{
    int rollNo;
    string name;
    float marks;
public:

    Student(): rollNo(0), name(""), marks(0){
    }

    Student(int rollNo, string name, float marks): rollNo(rollNo), name(name),
marks(marks){
    }

    friend class Faculty;                   // friend class
};

// friend class Faculty can access private members of Student class
class Faculty{
    public:

    void generateResult(Student& s){
        cout << s.rollNo << " | " << s.name << " | " << s.marks << endl;
    }

    void graceMarks(Student& s){
        s.marks += 5.5;
```

```
        cout << s.rollNo << " | " << s.name << " | " << s.marks << endl;
    }
};

int main(){

    Student s1(121,"shil", 40);

    Faculty f1;

    f1.generateResult(s1);  // faculty -> generating result
    f1.graceMarks(s1);      // faculty -> giving grace marks

    return 0;
}
```

Operator categories

- Arithmetic operators

```
 + - * / %
```

- Logical

```
   && || !
```

- Relational

```
< > <= >= == !=
```

- Assingment

```
= += -= *= /= %=
```

- Bitwise

```
& | << >> ~
```

- Unary (Increment/Decrement)
    - Pre/Post:

```
++ --
```

- Special Operator

```
>> << ? :  -> . :: .* [] () etc.
```

Operator that cannot be overloaded.

- `::` (scope resolution operator)
- `.` (member access operator)
- `?:` (ternary conditional operator)
- `sizeof` (size of operator)
- `typeid` (type identification operator)

Member vs non-member operators

Member operator overloading

- When an operator is overloaded as a member function, the left-hand operand of the operator is implicitly passed as the `this` pointer to the member function, and the right-hand operand is passed as an argument to the function.
- Member operator overloading is typically used when the operator needs to access the private members of the class or when the operator is a unary operator (e.g., increment, decrement).

```
// Member operator overloading
//op: +, -, *, /, etc.
returnType operator op (parameterList) { // Member operator overloading
    // Function body
}

// calling member operator overloading
className obj1, obj2;
obj1 op obj2; // obj1.operator op (obj2) for binary operator

// obj1.op(obj2); // equivalent to obj1.operator op (obj2)

op obj1; // obj1.operator op() for unary operator
example:
class Complex {
private:
    int real;
    int imag;
public:
    Complex(int r = 0, int i = 0) : real(r), imag(i) {}
    Complex operator + (const Complex& other) { // Member operator overloading for
addition
        return Complex(this->real + other.real, this->imag + other.imag);
```

```
    }
};
```

Non-member operator overloading

- When an operator is overloaded as a non-member function, both operands of the operator are passed as arguments to the function.
- Non-member operator overloading is typically used when the operator does not need to access the private members of the class or when the operator is a binary operator (e.g., addition, subtraction).

```
// Non-member operator overloading
// op: +, -, *, /, etc.
returnType operator op (const className& lhs, const className& rhs) { // Non-member operator overloading
    // Function body
}

example:
class Complex {
private:
    int real;
    int imag;
public:
    Complex(int r = 0, int i = 0) : real(r), imag(i) {}
    friend Complex operator + (const Complex& lhs, const Complex& rhs); // Non-member operator overloading for addition
};

// Definition of the non-member operator overloading function: friend function
Complex operator + (const Complex& lhs, const Complex& rhs) {
    return Complex(lhs.real + rhs.real, lhs.imag + rhs.imag);
}
```

Arithmetic operator overloading

- `+` (addition)
- `-` (subtraction)
- `*` (multiplication)
- `/` (division)
- `%` (modulus)

Relational operator overloading

- `<` (less than)
- `>` (greater than)
- `<=` (less than or equal to)
- `>=` (greater than or equal to)
- `==` (equal to)

- `!=` (not equal to)

> C++20 introduced the three-way comparison operator `<=>` (also known as the spaceship operator) which can be used to automatically generate all six relational operators based on a single implementation.

```
#include <iostream>
using namespace std;

class Complex{
    int real, imag;

public:
    Complex() : real(0), imag(0){
    }

    Complex(int real, int imag) : real(real), imag(imag){
    }

    int getImag() const {
        return imag;
    }

    void setImag(int imag) {
        this->imag = imag;
    }

    int getReal() const {
        return real;
    }

    void setReal(int real) {
        this->real = real;
    }

    void print(){
        cout << real << " + " << imag << "i" << endl;
    }

    //member function *this
//  Complex operator+(Complex& c){
//      return Complex(this->real + c.real, this->imag + c.imag);
//  }
    friend Complex operator+(Complex& first, Complex& second);
    friend Complex operator+(int first, Complex& second);

    bool operator==(Complex& c){
        return (this->real == c.real) && (this->imag == c.imag);
    }

    bool operator<(Complex& c){
        return (this->real < c.real) && (this->imag < c.imag);
```

```
    }

    friend bool operator<=(Complex& first, Complex& second);

    auto operator<=>(Complex& c) const = default;
};

Complex operator+(Complex& first, Complex& second){
    return Complex(first.real + second.real, first.imag + second.imag);
}

Complex operator+(int first, Complex& second){
    return Complex(first + second.real, second.imag);
}

// for <= operator non member function
bool operator<=(Complex& first, Complex& second){
    return (first.real < second.real) && (first.imag < second.imag);
}

int main() {

    int a = 10;
    int b = 20;

    int c = a + b;  // local variable: primitive datatypes

    cout << "Sum of Int: " << c << endl;

    Complex c1(10, 20);
    Complex c2(30, 40);

    c1.print();
    c2.print();

    Complex c3;

    c3 = c1 + c2; // Member
    //   c1.operator +(c2); //internally your compiler converts it for member
function
    // operator(&c1, c2);

    //Non-Member
    // operator+(c1,c2);

    c3.print();

    Complex c4 = 50 + c2;   //Non-Member

    c4.print();

    if(a == b)
        cout << "a is equal to b" << endl;
    else
```

```
        cout << "a is not equal to b" << endl;

    if(c1 == c2)
        cout << "Both complex numbers are equal." << endl;
    else
        cout << "Both complex numbers are not equal." << endl;

    if(c1 <= c2)
        cout << "first is less than or equal to second." << endl;
    else
        cout << "first is not less than or equal to second." << endl;

//  < > <= => == != : c++20 <=>

    return 0;
}
```